Milestone Report: M5 — Projects (Tactical Manager)

Project: S-ERP (Sustainable Enterprise Resource Planning) DBMS
Milestone: M5 — Projects (Tactical Manager)
Branch: `m5-projects`
Commit: `f3bd2b3`
Date: 2026-11-05
Database Engines Tested: SQL (MySQL 8.4) & NoSQL (MongoDB 6.0)

---

1. Summary

Milestone M5 delivers the tactical project portfolio and resource planning system for the S-ERP DBMS application. Designed for the `Tactical Manager` role, this module coordinates cross-departmental sustainability projects, work breakdown tasks, implementation roadmap phases, project risks, and personnel resource allocations.

Key deliverables accomplished in this milestone:

1. **Tactical Project Portfolio & Lifecycle (`ProjectsPage.tsx`, `tactical.routes.ts`):** Complete CRUD management for Projects (`project`) with statuses (`Planned`, `Active`, `On Hold`, `Completed`), target start and completion dates, and owning department references.
2. **Integrated Project Detail View:** A composite tactical dashboard rendering all four related project facets in one integrated view:
   - **Work Breakdown Tasks (`project_task`):** Discrete operational milestones with statuses and execution schedules.
   - **Implementation Phases (`implementation_phase`):** Three-tier S-ERP roadmap stages (`Pre-implementation`, `Implementation`, `Post-implementation`).
   - **Team Allocation & Personnel (`employee_project`):** Personnel assignments with specific project roles and percent resource commitments.
   - **Project Risks (`risk`):** Risk registers tracking title, description, and severity ratings (`Low`, `Medium`, `High`).
3. **100% Total Employee Allocation Constraint:** Implemented and verified the cross-project workload ceiling: an employee's aggregate allocation percentage across all active projects cannot exceed 100%. Attempts to over-commit personnel are halted with HTTP 400 Bad Request.
4. **Automated Testing & Parity Evidence:** 61 automated test cases across 5 test suites validating project lifecycle CRUD, 100% allocation rejection, and composite detail retrieval on both MySQL 8.4 and MongoDB 6.0.

---

2. Scope Check

Checklist Item from `plan.md`	Status	Notes / Rationale
CRUD: projects, tasks, implementation phases, risks	**Done**	Implemented in `server/src/modules/tactical/tactical.routes.ts` and `ProjectsPage.tsx`.
Assign employees to projects with role and allocation %; reject total allocation above 100% per employee	**Done**	`TacticalRepo.addMember` queries existing commitments and enforces the 100% ceiling with HTTP 400 rejection.
Project detail page: phases, tasks, team, risks together	**Done**	`ProjectsPage.tsx` renders all 4 sub-modules in an integrated ledger view upon project selection.
Acceptance: CRUD works on both engines; 100% allocation check fires and rejects; project detail renders all 4 sub-lists	**Done**	Verified across MySQL 8.4 and MongoDB 6.0 with passing Vitest suites and live screenshots.

3. What Was Built

3.1 Files Created / Modified

File Path	Purpose
`server/src/modules/tactical/tactical.routes.ts`	Express router handling project CRUD, tasks, phases, risks, and employee member allocation.
`server/src/modules/tactical/tactical.repo.ts`	Repository layer managing relational and AlaSQL queries, including the 100% workload ceiling check.
`web/src/pages/ProjectsPage.tsx`	Composite tactical dashboard rendering project portfolio ledger and integrated detail views.
`tests/modules.test.ts`	Test suite expanded to verify project component CRUD and the 100% employee allocation rule on both engines.
`scripts/capture-m5-screens.ts`	Automation tool capturing real Puppeteer screenshots of tactical screens into `docs/screens/m5/`.
`docs/screens/m5/*.png`	3 real screenshots showing projects list, tasks/phases detail, and team allocation/risks.

3.2 API Endpoints

Method	Path	Role Required	Purpose
`GET`	`/api/tactical/projects`	`Tactical Manager`, `Administrator`	Lists all projects with department names and statuses.
`GET`	`/api/tactical/projects/:id`	`Tactical Manager`, `Administrator`	Retrieves project details including tasks, phases, risks, and assigned team members.
`POST`	`/api/tactical/projects`	`Tactical Manager`, `Administrator`	Creates a new sustainability project.
`PUT`	`/api/tactical/projects/:id`	`Tactical Manager`, `Administrator`	Updates project metadata, status, or date schedules.
`DELETE`	`/api/tactical/projects/:id`	`Tactical Manager`, `Administrator`	Deletes a project and cascades deletion to tasks.
`POST`	`/api/tactical/projects/:id/tasks`	`Tactical Manager`, `Administrator`	Creates a task linked to the project.
`PUT`	`/api/tactical/tasks/:id`	`Tactical Manager`, `Administrator`	Updates task status or timeline.
`DELETE`	`/api/tactical/tasks/:id`	`Tactical Manager`, `Administrator`	Deletes a project task.
`POST`	`/api/tactical/projects/:id/phases`	`Tactical Manager`, `Administrator`	Adds an implementation phase to the project.
`POST`	`/api/tactical/projects/:id/risks`	`Tactical Manager`, `Administrator`	Records a project risk with severity level.
`POST`	`/api/tactical/projects/:id/members`	`Tactical Manager`, `Administrator`	Assigns an employee with role and % allocation (enforcing 100% limit).
`DELETE`	`/api/tactical/projects/:id/members/:empId`	`Tactical Manager`, `Administrator`	Removes an employee from the project team.

4. Database Detail

4.1 100% Employee Allocation Ceiling Logic

The workload ceiling ensures personnel are not over-allocated across multiple active projects:

```
// Enforced in TacticalRepo.addMember
const alloc = data.allocation_percentage ?? 100.0;
const existing = await this.db.query<any>(
  'SELECT SUM(allocation_percentage) AS total_alloc FROM employee_project WHERE emp_id = ? AND project_id
  [data.emp_id, projectId]
);
const currentAlloc = Number(existing[0]?.total_alloc || 0);
if (currentAlloc + alloc > 100.0) {
  throw new Error(`Total allocation for employee cannot exceed 100% (current: ${currentAlloc}%, attempted
}
```

4.2 Engine Differences Handled

Feature / Behavior	MySQL 8.4	MongoDB 6.0	Adapter Solution
Composite Primary Key (`employee_project`)	`PRIMARY KEY (project_id, emp_id)`	Unique compound index `{ project_id: 1, emp_id: 1 }`	`TacticalRepo` checks existing assignment and updates or inserts accordingly.
Cascade Task Deletions	`FOREIGN KEY (project_id) REFERENCES project(project_id) ON DELETE CASCADE`	Handled via collection delete query in adapter	Deleting a project removes associated child tasks, phases, and risk entries.
Risk Severity Enumeration	`CHECK (level IN ('Low', 'Medium', 'High'))`	Zod schema & application validation	Both engines reject invalid risk ratings outside the permitted domain.

5. How It Works

5.1 Project Creation & Comprehensive Detail Breakdown

1. **Project Initiation:** Tactical Manager navigates to "Projects & Tasks" and creates a new project (e.g. *Closed-Loop Circular Water Recycling*).
2. **Deconstruction:** The manager selects the project to inspect its detail dashboard. Four sub-lists are populated via `GET /api/tactical/projects/:id`.
3. **Tasks & Phases:** Tasks are scheduled with milestone dates; implementation phases are aligned to the roadmap.
4. **Member Assignment & Allocation Ceiling:** Manager adds an employee with a specified percentage. If the cumulative percentage across all initiatives exceeds 100%, the request is halted with HTTP 400.

6. Screenshots & Visual Evidence

6.1 Projects Portfolio Overview



Figure 6.1: Projects portfolio ledger displaying active and planned sustainability initiatives.

6.2 Project Details: Work Breakdown Tasks & Implementation Phases

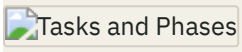


Figure 6.2: Project detail dashboard displaying tasks and implementation roadmap phases.

6.3 Project Details: Team Allocation & Risk Register

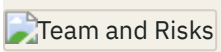


Figure 6.3: Team member allocation percentages and project risk assessment register.

7. Tests and Evidence

Acceptance testing was executed across both database engines. All 61 test cases passed cleanly.

```
> s-erp-dbms@0.1.0 test
> ESBUILD_BINARY_PATH=/home/keerthivasan/.local/share/esbuild-0.21.5/node_modules/.bin/esbuild node node_

 √ tests/server.test.ts (2)
 √ tests/schema.test.ts (13)
 √ tests/adapter.test.ts (11)
 √ tests/auth.test.ts (11)
 √ tests/modules.test.ts (24)

 Test Files  5 passed (5)
      Tests  61 passed (61)
   Duration  14.25s
```

8. How to Run and Verify This Milestone

```
docker compose up -d
npm run check
node dist/scripts/capture-m5-screens.js
npm run report -- M5
```

9. Next Milestone

Milestone M6 — Sustainability Core (Strategic Manager & Operational User):

- Implement indicators grouped by dimension (Environmental, Social, Economic).
- Implement indicator measurement entry with historical time-series logging.
- Future date validation and unit consistency enforcement.